

LABORATORY OBSERVATION

Course: Full Stack Web Development

Course Code: 23CM4121

Observation Task No.: 4

Student Name: T. Akshaya

Roll No.: A24126552122

Date.:

1. TITLE

To-Do List Application — JavaScript DOM Manipulation and Event Listeners

2. AIM

To develop a to-do list application using JavaScript, DOM manipulation, and event listeners that allows the user to add, complete, and delete tasks dynamically.

3. OBJECTIVE

- To dynamically create and display task items using DOM manipulation.
 - To use event listeners to handle Add, Complete, and Delete actions.
 - To visually indicate a completed task using a CSS class.
 - To remove a task from the list when it is deleted.
 - To display an appropriate message when the task list is empty.
-

4. THEORY

DOM manipulation allows JavaScript to create, modify, and remove elements on a webpage after it has loaded. New elements such as a task's `<li>`, `<span>`, and buttons can be created using `document.createElement()` and added to the page using `appendChild()`, while `textContent` sets the visible text inside them.

An event listener, added using `addEventListener()`, runs a function whenever a specific action occurs, such as clicking the Add, Complete, or Delete button, or pressing a key in the input field. The `classList.toggle()` method adds or removes a CSS class depending on whether it is already present, which is used here to mark a task as completed. The `remove()` method deletes an element from the page entirely, which is used to delete a task, while checking the number of items in the list controls whether the "No tasks available" message is shown or hidden.

5. ALGORITHM / PROCEDURE

1. Create a new HTML file and declare `<!DOCTYPE html>` at the top.
2. Add a `<style>` section for CSS styling.
3. Create a container with a heading, a text input, and an "Add Task" button.

4. Add an empty `<ul>` for the task list and a paragraph for the "No tasks available" message.
 5. Select the input, button, list, and message elements using `getElementById()`.
 6. Define a function `updateEmptyMessage()` that shows or hides the empty message based on the number of tasks.
 7. Add a click event listener to the "Add Task" button.
 8. Inside the listener, read and trim the input value, and show an alert if it is empty.
 9. Create a new `<li>` element with a `<span>` to hold the task text.
 10. Create a button group with "Complete" and "Delete" buttons and add them to the `<li>`.
 11. Add a click event listener on the Complete button to toggle a "completed" class on the task.
 12. Add a click event listener on the Delete button to remove the task and update the empty message.
 13. Append the new task to the task list, clear the input field, and call `updateEmptyMessage()`.
 14. Add a keypress event listener on the input so pressing Enter also adds the task.
 15. Save the file and open it in a browser to test.
 16. Test adding, completing, and deleting tasks, and record the results as test cases.
-

6. TASK DESCRIPTION

A to-do list application was developed using JavaScript, DOM manipulation, and event listeners, covering the following:

- Provided a text box for entering a task.
 - Provided an "Add Task" button.
 - When the button is clicked, a new task item is dynamically created and added to the list.
 - Each task contains a "Complete" button; clicking it visually indicates that the task has been completed.
 - Each task also contains a "Delete" button; clicking it removes the corresponding task from the webpage.
 - An appropriate message ("No tasks available.") is displayed when the task list is empty.
-

7. PROGRAM

Task 4.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>To-Do List</title>
<style>
body {
  font-family: 'Segoe UI', Arial, sans-serif;
  background: #f0f4f8;
  display: flex;
  flex-direction: column;
  align-items: center;
  padding: 40px;
```

```
}
.container {
  background: #fff;
  padding: 25px 30px;
  border-radius: 10px;
  box-shadow: 0 2px 10px rgba(0,0,0,0.1);
  width: 380px;
}
h2 {
  text-align: center;
  color: #2c3e50;
  margin-top: 0;
}
.input-row {
  display: flex;
  gap: 8px;
  margin-bottom: 15px;
}
.input-row input {
  flex: 1;
  padding: 8px;
  border: 1px solid #ccc;
  border-radius: 5px;
}
.input-row button {
  padding: 8px 14px;
  background: #2c7be5;
  color: #fff;
  border: none;
  border-radius: 5px;
  cursor: pointer;
}
.input-row button:hover {
  background: #1a5cbf;
}
#taskList {
  list-style: none;
  padding: 0;
  margin: 0;
}
#taskList li {
  display: flex;
  align-items: center;
  justify-content: space-between;
  background: #f8f9fb;
  padding: 8px 12px;
  border-radius: 6px;
  margin-bottom: 8px;
  border-left: 5px solid #2c7be5;
}
```

```
#taskList li.completed span {
  text-decoration: line-through;
  color: #888;
}
```

```
#taskList li.completed {
  border-left-color: #2ecc71;
}
```

```
.task-text {
  flex: 1;
  word-break: break-word;
}
```

```
.btn-group {
  display: flex;
  gap: 6px;
  margin-left: 8px;
}
```

```
.complete-btn, .delete-btn {
  border: none;
  padding: 5px 10px;
  border-radius: 4px;
  color: #fff;
  cursor: pointer;
  font-size: 13px;
}
```

```
.complete-btn {
  background: #2ecc71;
}
```

```
.complete-btn:hover {
  background: #27ae60;
}
```

```
.delete-btn {
  background: #e74c3c;
}
```

```
.delete-btn:hover {
  background: #c0392b;
}
```

```
#emptyMsg {
  text-align: center;
  color: #888;
  margin-top: 10px;
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div class="container">
```

```
<h2>My To-Do List</h2>
```

```
<div class="input-row">
```

```
<input type="text" id="taskInput" placeholder="Enter a task...">
```

```
<button id="addBtn">Add Task</button>
```

```
</div>
<ul id="taskList"></ul>
<p id="emptyMsg">No tasks available.</p>
</div>

<script>
const taskInput = document.getElementById('taskInput');
const addBtn = document.getElementById('addBtn');
const taskList = document.getElementById('taskList');
const emptyMsg = document.getElementById('emptyMsg');

function updateEmptyMessage() {
  emptyMsg.style.display = taskList.children.length === 0 ? 'block' : 'none';
}

addBtn.addEventListener('click', function () {
  const taskText = taskInput.value.trim();

  if (taskText === "") {
    alert('Please enter a task.');
```

return;

}

```
const li = document.createElement('li');

const span = document.createElement('span');
span.classList.add('task-text');
span.textContent = taskText;

const btnGroup = document.createElement('div');
btnGroup.classList.add('btn-group');

const completeBtn = document.createElement('button');
completeBtn.textContent = 'Complete';
completeBtn.classList.add('complete-btn');

const deleteBtn = document.createElement('button');
deleteBtn.textContent = 'Delete';
deleteBtn.classList.add('delete-btn');

completeBtn.addEventListener('click', function () {
  li.classList.toggle('completed');
});

deleteBtn.addEventListener('click', function () {
  li.remove();
  updateEmptyMessage();
});

btnGroup.appendChild(completeBtn);
```

```

    btnGroup.appendChild(deleteBtn);
    li.appendChild(span);
    li.appendChild(btnGroup);
    taskList.appendChild(li);

    taskInput.value = "";
    taskInput.focus();
    updateEmptyMessage();
  });

  taskInput.addEventListener('keypress', function (e) {
    if (e.key === 'Enter') {
      addBtn.click();
    }
  });

  updateEmptyMessage();
</script>

</body>
</html>

```

8. CODE EXPLANATION

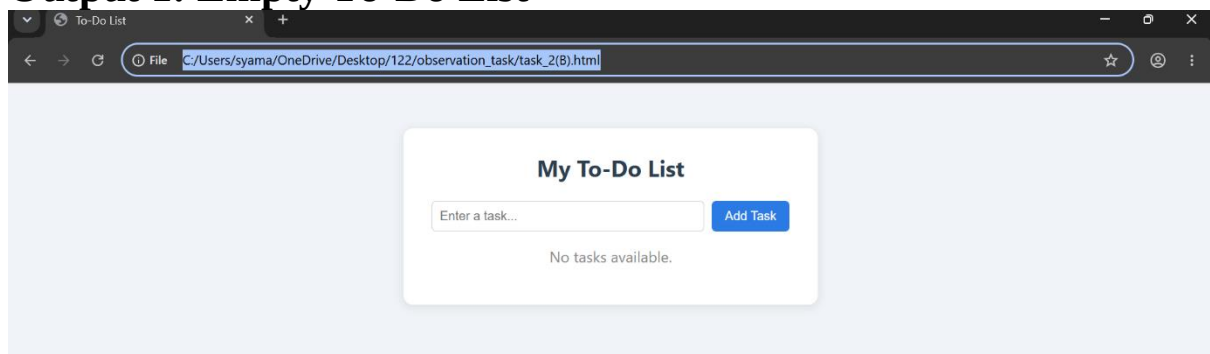
Tag / Function	Purpose
getElementById()	Selects an HTML element (input, button, list, or message) by its id.
addEventListener('click', fn)	Attaches a function to an element that runs whenever it is clicked.
.value.trim()	Reads the text typed into the input field and removes extra spaces.
alert()	Displays a popup message, used here to warn if the input is empty.
document.createElement()	Creates a new HTML element (like , , <div>, or <button>) in memory.
classList.add()	Adds a CSS class to an element, used to style the task text and buttons.
classList.toggle()	Adds a class if it is not present, or removes it if it is — used to mark a task completed or not.
textContent	Sets the visible text inside an element, used for the task text and button labels.
appendChild()	Adds a created element as a child of another element, building up each task item.
remove()	Removes an element from the page, used to delete a task.
style.display	Shows or hides the "No tasks available" message based on whether the list is empty.
keypress event / e.key === 'Enter'	Detects when the Enter key is pressed in the input field, so the task can be added without clicking the button.
<input>, <button>, ,	Form and list tags used to collect the task text and display each task item.

9. TEST CASES AND EXECUTION DATA

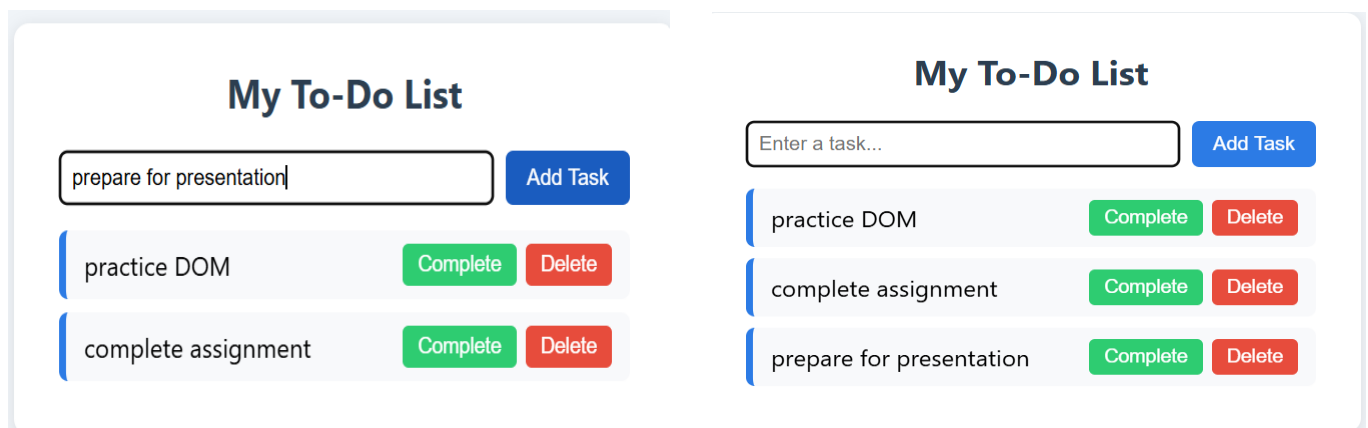
Test Case	Action	Expected Result	Actual Result	Status
TC-01	Click "Add Task" with an empty input	Alert message is shown, no task is added	Alert shown correctly, no task added	Pass
TC-02	Enter a task and click "Add Task"	New task appears in the list	Task added and displayed correctly	Pass
TC-03	Click "Complete" on a task	Task text shows a strikethrough and the border colour changes	Task marked completed correctly	Pass
TC-04	Click "Delete" on a task	Task is removed from the list	Task removed correctly	Pass
TC-05	Delete all tasks	"No tasks available" message is displayed	Empty message displayed correctly	Pass

10. OUTPUT

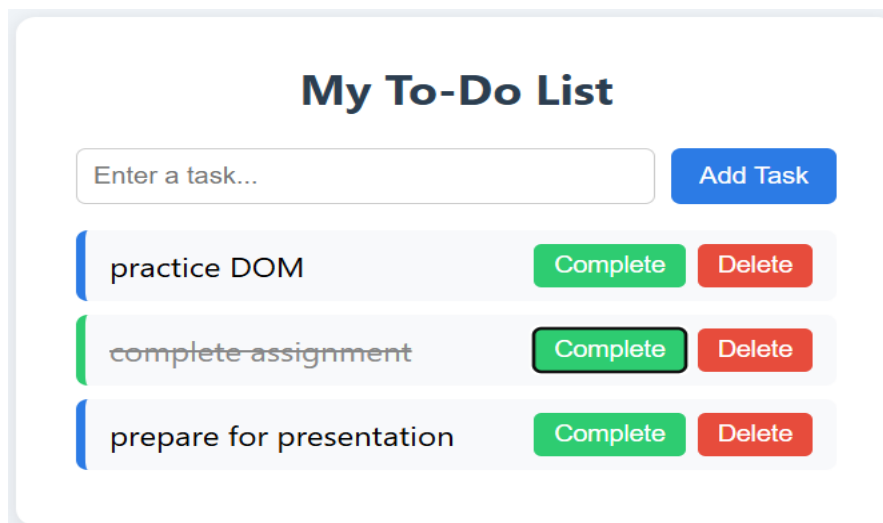
Output 1: Empty To-Do List



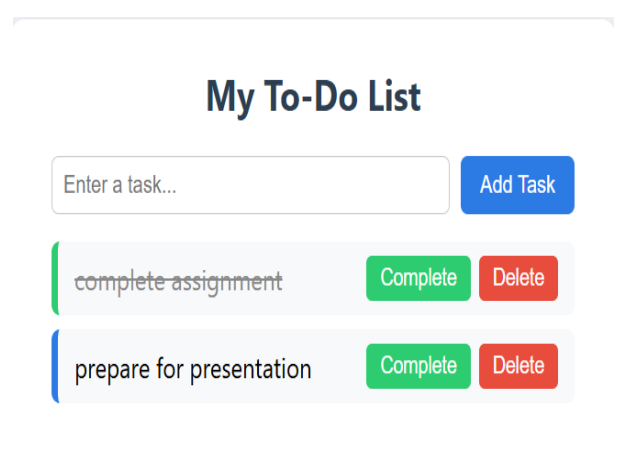
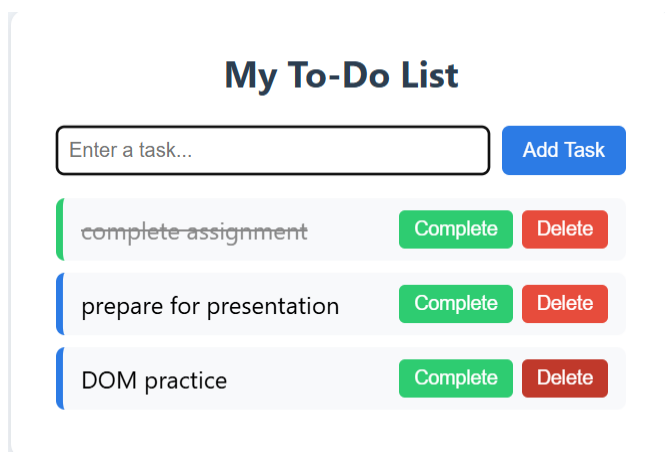
Output 2: Task Added to the List



Output 3: Task Marked as Completed



Output 4: Task Deleted from the List



11. RESULT

Thus, a to-do list application was successfully developed using JavaScript, DOM manipulation, and event listeners. Tasks could be added, marked as completed, and deleted dynamically, and the empty-list message displayed correctly when no tasks were present. All features were tested in a web browser and found to work correctly.